

# Software Testing Fundamentals

- Overview of Testing Concepts, Levels, Types, and Techniques

# What is Software Testing?

- **Software Testing** is the process of checking a software application to find errors (bugs) and ensure that it works correctly according to the given requirements.
- In simple words:  
It is used to verify that the software is **working properly and giving correct output**.
- **Example:**  
If you create a login page, testing checks whether:
  - Correct password allows login
  - Wrong password shows error
  - System does not crash

# Why is Software Testing Important?

## **1. Finds Errors (Bugs)**

Testing helps identify mistakes before users use the software.

## **2. Ensures Quality**

It ensures the software meets requirements and works correctly.

## **3. Improves Security**

Testing helps find security issues like data leaks or unauthorized access.

## **4. Better User Experience**

Ensures smooth working, fast response, and no crashes.

## **5. Saves Cost**

Fixing bugs early is cheaper than fixing after release.

## **6. Improves Performance**

Checks speed, responsiveness, and stability of the system.

# Verification

- **Definition:**

Verification is the process of checking whether the software is being built correctly according to design and requirements.

**Simple line:**

**“Are we building the product right?”**

- **Focus:** Process (documents, design, code)
- **When done:** Before execution (no actual running required)
- **Techniques:**
  - Reviews
  - Inspections
  - Walkthroughs
- **Example:**

Checking design documents of a login system to ensure all requirements are included.

# Validation

**Definition:**

Validation is the process of checking whether the final software meets user needs and works correctly.

**Simple line:**

**“Are we building the right product?”**

**Focus:** Product (actual working software)

**When done:** After development (execution required)

**Techniques:**

- Testing (functional, system testing)

**Example:**

Testing login system by entering username & password to see if it works properly.

# Key Differences Table

| Feature   | Verification                 | Validation                         |
|-----------|------------------------------|------------------------------------|
| Meaning   | Check correctness of process | Check correctness of product       |
| Question  | Are we building it right?    | Are we building the right product? |
| Execution | No need to run code          | Requires running code              |
| Focus     | Documents & design           | Final software                     |
| Example   | Design review                | Login testing                      |

Verification = Checking (before coding/testing)

Validation = Testing (after development)

# STLC

- **Requirement Analysis**
- **Test Planning**
- **Test Case Design**
- **Test Environment Setup**
- **Execution**
- **Defect Reporting**
- **Closure**

# Phases of STLC with Example

- **1. Requirement Analysis**
- In this phase, testers understand what needs to be tested.
- **Example: (Login Page)**
- There should be a username field
- There should be a password field
- There should be a login button
- Tester checks:
  - ✓ Requirements are clear
  - ✓ Requirements are testable



- **2. Test Planning**
- In this phase, a testing plan is created.
- Includes:
  - Number of testers
  - Testing tools (manual or automation)
  - Time required
- **Example:**
  - 2 testers assigned
  - Testing duration: 3 days
  - Manual testing will be used

- **3. Test Case Design**
- Test cases are written in this phase.
- Example

| Test Case ID | Description                     | Expected Result         |
|--------------|---------------------------------|-------------------------|
| TC1          | Enter valid username & password | Login successful        |
| TC2          | Enter invalid password          | Error message displayed |
| TC3          | Leave fields empty              | Warning message shown   |

- **4. Test Environment Setup**
- The environment required for testing is prepared.
- **Example:**
- Install browser (Chrome)
- Setup server
- Prepare test database

- **5. Test Execution**
- Test cases are executed in this phase.
- **Example:**
- TC1 → Passed
- TC2 → Failed (no error message shown)
- A defect is found

- **6. Defect Reporting**

- Bugs are reported to the development team.
- **Example:**
- Issue: No error message for wrong password
- Status: Open
- Assigned to developer

- **7. Test Closure**
- Testing process is completed and reports are prepared.
- Includes:
  - Test summary report
  - Number of passed/failed test cases
  - Lessons learned
- **Example:**
  - Total test cases: 10
  - Passed: 8
  - Failed: 2

# Levels of Testing

- Unit Testing
- Integration Testing
- System Testing
- Acceptance Testing

# 1. Unit Testing

Smallest part of the software is tested (individual function/module).

- Done by: Developers
- Focus: Single unit or function

## **Example:**

A function:

`add(a, b)`

Test cases:

• `add(2,3) → 5` 

• `add(-1,1) → 0` 

✓ Only one small part is tested



## 2. Integration Testing

- Multiple units/modules are combined and tested together.
- Done by: Developers/Testers
- Focus: Interaction between modules
- **Example:**  
Login system:
  - Module 1: Login page
  - Module 2: Database
  - Test:
    - ✓ Username entered → goes to database → correct response
    - If data is not fetched properly → defect ✗

# 3. System Testing

- Entire system is tested as a whole.
- Done by: Testers
- Focus: Complete functionality
- **Example:**  
Full application testing:
  - Login
  - Dashboard
  - Logout
  - ✓ Check everything works together correctly

# 4. Acceptance Testing

- Final testing to check if system meets user requirements.
- Done by: Client/User
- Focus: Business requirements
- **Example:**  
Client checks:
  - ✓ Can user login easily?
  - ✓ Is system working as per requirement?
- If client is satisfied → Software accepted 

# Types of Testing

- Manual Testing
- Automation Testing
- Functional Testing
- Non-Functional Testing

# 1. Manual Testing

- Testing done by humans without using tools.
- Tester manually checks the application
- No automation scripts
- **Example:**  
Tester opens login page and:
  - Enters username & password
  - Clicks login
  - Checks result manually
  - ✓ Simple but time-consuming

## 2. Automation Testing

- Testing done using tools/scripts.
- Faster than manual testing
- Used for repetitive tasks
- **Example:**  
Using tools like:
  - Selenium
  - Script automatically:
    - Opens browser
    - Enters login details
    - Checks output
    - ✓ Saves time and effort

# 3. Functional Testing

- Checks **what the system does** (features).
- Focus on functionality
- **Example:**  
Login page:
  - Valid login → success
  - Invalid login → error message
  - ✓ Checks expected output

## 4. Non-Functional Testing

- Checks **how the system performs**.
- Focus on performance, speed, usability
- **Example:**
- Website load time < 3 seconds
- App handles 1000 users at once
- ✓ Checks performance and quality



# Testing Techniques

- Black Box
- White Box
- Grey Box
- Examples included

# 1. Black-box Testing

- Tester does **NOT** know internal code  
Focus is only on **input and output**
- No knowledge of coding required
- Tests functionality
- **Example (Login Page):**
- Input: Username = "admin", Password = "1234"
- Output: Login successful 
- Input: Wrong password
- Output: Error message 
- ✓ Tester only checks result, not code

## 2. White-box Testing

- Tester knows **internal code and logic**  
Focus on **code structure**
- Done by developers
- Requires programming knowledge
- **Example:**  
Code:

- `if password == "1234":`
- `print("Login successful")`
- `else:`
- `print("Error")`
- Test cases:
- Check all conditions (if / else)
- Check loops, conditions, paths
- ✓ Tester checks internal logic

# 3. Grey-box Testing

- Tester has **partial knowledge of code**
- Combination of black-box + white-box
- Knows some internal logic but tests like user
- **Example:**  
Tester knows:
  - Data comes from database
  - Test:
    - Enter username
    - Check if correct data is fetched from DB
  - ✓ Both internal idea + external testing

# Real-Life Example

- Think of a **TV remote**
- Black-box: Press button → check TV works
- White-box: Know internal circuit of remote
- Grey-box: Know battery + basic working

# Test Case Design Techniques

- These techniques help testers create **effective and smart test cases**.
- Main techniques:
- Boundary Value Analysis (BVA)
- Equivalence Partitioning (EP)

# 1. Boundary Value Analysis (BVA)

- Focus on **boundary values (edges)** of input range  
Errors mostly occur at boundaries
- **Condition:**  
Boundary values should be between **18 to 60**
- Boundary values:
- Minimum: 18
- Maximum: 60



| Test Case | Input | Expected Result |
|-----------|-------|-----------------|
| TC1       | 17    | Invalid ❌       |
| TC2       | 18    | Valid ✅         |
| TC3       | 19    | Valid ✅         |
| TC4       | 59    | Valid ✅         |
| TC5       | 60    | Valid ✅         |
| TC6       | 61    | Invalid ❌       |

✓ Focus = edges (17,18,19 and 59,60,61)

## 2. Equivalence Partitioning (EP)

- Divide input data into **groups (partitions)**  
Test only one value from each group
- **Condition:**  
Marks should be between **0 to 100**
- 👉 Partitions:
- Invalid:  $< 0$
- Valid: 0–100
- Invalid:  $> 100$

| Test Case | Input | Expected Result |
|-----------|-------|-----------------|
| TC1       | -5    | Invalid ❌       |
| TC2       | 50    | Valid ✅         |
| TC3       | 120   | Invalid ❌       |

✓ Instead of testing all values, test **one from each group**

# Writing Effective Test Plan

- Test Plan = Document that defines **how testing will be done**
- **Contents of Test Plan:**
  - Objective (what to test)
  - Scope (features to test)
  - Testing strategy
  - Resources (team)
  - Tools
  - Schedule
  - Risks

# Example (Login System):

- Objective: Test login functionality
- Scope: Username, password validation
- Tool: Manual testing
- Time: 3 days
- Team: 2 testers

# Writing Effective Test Cases

- Test Case = Step-by-step process to test a feature
- Good Test Case Format:

| Test Case ID | Description   | Steps                             | Expected Result  |
|--------------|---------------|-----------------------------------|------------------|
| TC1          | Valid login   | Enter correct username & password | Login successful |
| TC2          | Invalid login | Enter wrong password              | Error message    |
| TC3          | Empty fields  | Leave fields blank                | Warning shown    |

# Characteristics of Good Test Cases

- ✓ Clear and simple
- ✓ Easy to understand
- ✓ Covers all scenarios
- ✓ Includes expected result